

No. of Functional Features Included in the Solution

Date	27/06/2026
Team ID	Not Assigned
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	5 Marks

Functional Features Overview:

This document lists all the functional features included in the Rising Waters flood prediction project solution. Each feature is described with its purpose, implementation status, and contribution to the overall solution.

S.No	Feature Name	Feature Description	Module / Component	Status (Done / In Progress / Pending)	Marks Contribution
1	Exploratory Data Analysis (EDA)	Performs univariate analysis (distribution plots, box plots), multivariate analysis (correlation heatmap, violin plots) and descriptive statistics on the flood dataset to understand feature patterns and class distribution.	flood_eda.ipynb	Done	1 Mark
2	Data Preprocessing Pipeline	Handles missing values using median imputation, detects and caps outliers using IQR method, encodes categorical variables using Label Encoding, applies 80/20 stratified train-test split and normalises features using StandardScaler.	flood_model.ipynb	Done	1 Mark
3	Multi-Model Training & Comparison	Trains and evaluates four classification algorithms — Decision Tree, Random Forest, KNN, and XGBoost — using confusion matrix, classification report, F1-score and ROC-AUC metrics. Compares all models side by side.	flood_model.ipynb	Done	1 Mark
4	XGBoost Best Model Selection & Saving	Selects XGBoost as the best model with 96.55% accuracy. Handles class imbalance using scale_pos_weight. Saves the trained model as floods.save and the fitted StandardScaler as transform.save using Joblib for deployment.	flood_model.ipynb	Done	0.5 Mark
5	Flask Web Application	Builds a Flask web application with 4 routes: / (home.html), /Predict (index.html), /chance (chance.html), /no_chance (no_chance.html). Loads floods.save and transform.save at startup for real-time inference.	app.py	Done	0.5 Mark
6	Real-Time Flood Prediction UI	Provides an input form (index.html) where users enter 10 meteorological parameters. The Flask backend scales inputs using transform.save, predicts using floods.save, and routes to the appropriate result page with probability score.	index.html app.py	Done	0.5 Mark

S.No	Feature Name	Feature Description	Module / Component	Status (Done / In Progress / Pending)	Marks Contribution
7	Result Pages with Advisory Actions	Displays flood prediction result on chance.html (flood detected) or no_chance.html (no flood) with flood probability percentage, animated progress bar, input summary and actionable advisory recommendations.	chance.html no_chance.html	Done	0.25 Mark
8	Responsive Dark UI with Animation	Implements a dark-themed responsive web interface using main.css with animated rain canvas effect (main.js), probability bar animation, glowing input fields, mobile-responsive grid layout and professional typography.	main.css main.js home.html	Done	0.25 Mark
9	Cloud Deployment on Render.com	Deploys the Flask application on Render.com free tier using Procfile (web: gunicorn app:app). Application is accessible globally at https://rising-waters-flood-prediction.onrender.com with HTTPS and auto-restart.	Render.com Procfile	Done	Bonus
10	GitHub Version Control & CI/CD	Full project codebase committed to GitHub repository. Render.com is linked to GitHub for continuous deployment — every push to the main branch triggers automatic redeployment of the live application.	GitHub Repository	Done	Bonus

Feature Summary:

Metric	Count / Value
Total Features Planned	10
Total Features Implemented	10
Core / Must-Have Features	8 (EDA, Preprocessing, 4 Models, Model Saving, Flask App, Prediction UI)
Additional / Nice-to-Have Features	2 (Cloud Deployment on Render.com, GitHub CI/CD Pipeline)
Features Tested & Verified	10 / 10 — All features tested locally and on live Render.com deployment

Feature Category Breakdown:

S.No	Category	Features in Category	Example Features
1	User Interface (UI)	3	Responsive dark-themed home page, prediction input form with range hints and unit tags, result pages with probability gauge and advisory actions
2	Backend / Logic	4	Flask routing (4 routes), form validation and error handling, StandardScaler preprocessing, XGBoost model inference and probability calculation
3	Database / Storage	2	floods.save (serialised XGBoost model), transform.save (fitted StandardScaler) — both loaded by Flask at startup using Joblib
4	API / Integration	2	Render.com cloud deployment via Procfile and Gunicorn, GitHub repository with automated CI/CD deployment pipeline on push to main branch
5	Security / Authentication	1	No authentication required per project scope. HTTPS enforced by Render.com. No personal user data collected or stored. Input validation via try-except in app.py